

RTFVE: Realtime Face Video Enhancement

Varun Ramesh Jois[✉], Antonella DiLillo, James Storer

Brandeis University, Waltham MA 02453, USA
{vjois,dilant,storer}@brandeis.edu

Abstract. There’s been a surge in adoption of video conferencing applications for both personal and business use cases. However, the bandwidth limitations faced by many users worldwide may restrict the optimal use of such applications. Although deep learning offers a solution for enhancing low bit rate videos, most models today are either hard to incorporate with modern compression standards or require specialized hardware to run such as significant GPUs making these models impractical. To address these issues, we introduce the Realtime Face Video Enhancement (RTFVE) model which can be easily incorporated with any video decoder and can run in realtime on ordinary CPUs. Experiments show that our model improves perceptual quality over the compressed video baseline at multiple low bitrate settings. The source code will be made available at <https://github.com/varun-jois/RTFVE>.

Keywords: Face Enhancement · Realtime · Videoconferencing.

1 Introduction

Over the last decade videocalling has become one of the most popular forms of communication. With today’s technology, everyone has access to a phone with a front-facing camera and an internet connection, often making video calling a daily occurrence for many users. Ever since the lockdowns during the COVID-19 pandemic, the adoption of this technology has grown exponentially with many new companies providing services for it.

While there has been a global trend towards faster internet speeds, there are still millions of people around the world that experience slow internet connections. And while the video compression standards are flexible with handling low bandwidth settings, they come with the cost of low quality video quite often making videocalls untenable. This being the case even though many of these users have the basic hardware to handle a standard videocall.

Deep learning based methods hold promise when it comes to improving the perceptual quality of faces. There have been advances in a myriad of fields such as face restoration, face deblurring, face super-resolution, etc. However, when it comes to face enhancement for videocalls there have been two problems stymieing the adoption of deep learning: 1) Models that cannot be easily integrated into the ubiquitous compression standards and 2) Models that require significant GPU and NPU resources that typical users don’t have. In this paper, we address both of these issues.

Table 1. Relative speed comparison of various methods ranging from classical computer vision algorithms to current state-of-the-art face restoration models on a low cost CPU. For realtime performance an FPS of at least 24 is required.

Model	Type	Frames per Second ($\uparrow$)
Non-Local Means Denoising [3]	Classical	4.56
Bilateral Filtering [17]	Classical	13.89
Split-Bregman [6]	Classical	17.71
Chambolle [4]	Classical	7.60
Codeformer [23]	Neural Network	0.05
GPEN [21]	Neural Network	2.47
GFP-GAN [19]	Neural Network	2.02
RTFVE (Ours)	Neural Network	24.85

Our contributions are as follows:

1. We present our model **RealTime Face Video Enhance-ment** (RTFVE) a model for face enhancement that can easily be integrated with the decoder of a video compression standard.
2. For the bandwidth cost of a handful of high-quality reference frames, our model is able to improve the visual quality of faces over the compression standards running in realtime on typical CPUs (e.g. laptops).

2 Related Work

2.1 Face Restoration

Face restoration is the task of producing a high quality image of a face from its low quality counterpart. GPEN [21] works by embedding a trained GAN-prior-network as the decoder of a U-shaped DNN and then fine-tuning the DNN with synthesized low quality face images. CodeFormer [23] looks at the blind face restoration task as a codebook prediction task. They first learn a discrete codebook and decoder to store high quality parts of faces. With the codebook and decoder fixed, they train a transformer based model to predict the code sequence of the low quality input image thereby modeling the properties of the low quality faces. PGDiff [20] introduces the concept of partial guidance to the diffusion process by modeling properties such as face structure and color statistics and applying this guidance during the reverse diffusion process. FRR-Net [14] is a face restoration and lighting network that incorporates a distortion classifier to use the class as a prior and dice loss for segmentation masks to only focus on the face region. They propose a new degradation scheme that also include illumination distortions.

2.2 Video Restoration

MFQE [7] leverages a BiLSTM based detector to locate Peak Quality Frames (PQFs) in compressed video and then uses a Multi-Frame Convolutional Neural

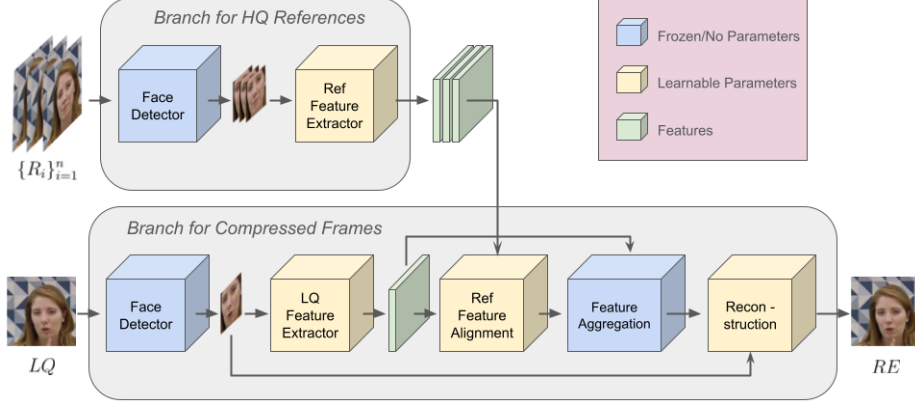


Fig. 1. The Realtime Face Video Enhancement (RTFVE) model. Our model takes the compressed, low-quality frames LQ from a video call, and n high-quality reference frames $\{R_i\}_{i=1}^n$ to produce the enhanced reconstructed frame RE .

Network to enhance the quality of compressed video using neighboring PQFs. They use multi scale features and dense connections to improve enhancement performance. In EDVR [18] the authors perform feature alignment using a pyramid, cascading, deformable convolution block (PCD). They then combine the aligned frame features using the temporal and spatial attention module (TSA) to perform attention both temporally and spatially before finally reconstructing the output. BasicVSR [5] uses bidirectional propagation, flow-based alignment, concatenation based aggregation and pixel shuffle to perform restoration. [1] uses keyframes as references and computes multiscale features which are used to produce the final output.

3 RTFVE Framework

Our model is inspired by previous work that leverages reference images such as [9,10] for Super-Resolution and [1] for Face Video Enhancement. The objective of our model is to leverage a handful of high quality reference images to improve the aesthetic of a low quality video stream. An equally important objective of ours is to make our model practical in terms of latency and hardware requirements. A model that is not at least 24 frames per second may diminish the user experience and a model that requires substantial GPU power to run may not be practical for the limited resources of many users.

Our model, named Realtime Face Video Enhancement (RTFVE) is depicted in Figure 1. It is comprised of five parts; a face detector, feature extraction blocks, an alignment module, a feature aggregation module and a reconstruction module. The model takes a low quality image LQ and n high quality reference images $\{R_i\}_{i=1}^n$ as inputs to produce the enhanced reconstructed frame RE .

3.1 Face Detector

The first part of our model is the face detector f_d which locates and crops the face in the low quality and reference images and passes them along to the rest of the model.

$$\begin{aligned} LQ^{\text{face}} &= f_d(LQ) \\ R_i^{\text{face}} &= f_d(R_i) \quad \text{for } i = 1 \dots n \end{aligned} \quad (1)$$

By working on just the faces, we can considerably reduce the resolution of our inputs thereby reducing latency. In our model, we select a 256×256 size crop. For our work, we used an off the shelf face detector [2] that we fixed. We also add a gaussian blur to the low quality image using a kernel of shape 3×3 and $\sigma = 0.8$.

3.2 Feature Extractor and the Shuffle Unit

After obtaining the face crops, we first perform pixel unshuffling [16] to reduce the resolution before passing them to the feature extractors f_e . This is the reverse of the shuffling operation mentioned in [16] that is a popular method for reducing the resolution size without losing any data by increasing the number of channels. By working in lower resolution space, we greatly increase speed of the model. Here we reduce the resolution by $4\times$. We use different extractors for the LQ and reference images.

$$\begin{aligned} F_{LQ} &= f_e^{LQ}(LQ^{\text{face}}) \\ F_{R_i} &= f_e^R(R_i^{\text{face}}) \quad \text{for } i = 1 \dots n \end{aligned} \quad (2)$$

Feature extraction is a necessary step for neural network architectures and deeper features offer better semantic representation. But deep models are computationally expensive making it unfeasible for low latency. To overcome this we designed the extractors, as well as the other modules using shuffle units [12]. These units are analogous to the units in ResNet [8] with three convolutional layers per unit. However, they greatly differ in the number of parameters and speed since the former performs depthwise group convolutions and 1×1 convolutions instead of dense convolutions. The shuffle unit also performs a channel split operation working on half the number of channels in any block thereby further increasing speed. For our features extractors, we used 10 shuffle units giving us deeper features and speed.

3.3 Feature Alignment

To get the most information out of the reference features, they need to be aligned with the low quality image features. We perform the alignment using spatial transformer alignment [10] f_a :

$$F_{R_iA} = f_a(F_{LQ}, F_{R_i}) \quad \text{for } i = 1 \dots n \quad (3)$$

This module works by aligning the features of the high quality reference images $\{F_{R_i}\}_{i=1}^n$ with the features of the low quality frame F_{LQ} in feature space. The original alignment module had the major drawback of two fully connected layers. This drastically increased the number of parameters and computation time. To remedy this, we used downsampling shuffle units within the localization network so the number of parameters given to the fully connected layers is low.

Table 2. Performance scores with respect to video compression codec and compression rate. Higher the CRF, more the compression.

Codec	CRF	Model	PSNR ($\uparrow$)	SSIM ($\uparrow$)	LPIPS ($\downarrow$)
H.264	36	H.264 (baseline)	33.0196	0.9089	0.0826
		RTFVE	33.5172	0.92	0.1073
		RTFVE-gan	33.3745	0.9118	0.0715
	40	H.264 (baseline)	30.833	0.8752	0.1273
		RTFVE	31.6259	0.8961	0.1365
		RTFVE-gan	31.2276	0.8799	0.1076
	44	H.264 (baseline)	28.566	0.8307	0.1872
		RTFVE	29.3205	0.8586	0.1851
		RTFVE-gan	29.0675	0.8433	0.1447
H.265	36	H.265 (baseline)	33.0416	0.9131	0.0779
		RTFVE	33.7089	0.9271	0.0926
		RTFVE-gan	33.1195	0.9086	0.0899
	40	H.265 (baseline)	30.7886	0.8788	0.1226
		RTFVE	31.5147	0.8993	0.133
		RTFVE-gan	31.0642	0.8813	0.1106
	44	H.265 (baseline)	28.5467	0.8353	0.1899
		RTFVE	29.2606	0.8631	0.1876
		RTFVE-gan	28.9606	0.8468	0.1434

3.4 Feature Aggregation

With the aligned reference features $\{F_{R_iA}\}_{i=1}^n$ in hand, we can now aggregate our references to obtain the most useful signals. We adopt the weighted aggregation module f_{agg} in [10] as is: since it's parameter free and efficient.

$$F_{agg} = f_{agg}(F_{LQ}, \{F_{R_iA}\}_{i=1}^n) \quad (4)$$

This module works by giving a greater weight to the reference features that are closest (in terms of L2 distance) to the features of the low-quality frame. This weighting is done for all regions in feature space and so one feature map can be given a greater weight at one region whereas another feature map can be given a greater weight in another region. It should be noted that this module does not contain parameters as it is based on the L2 distance and the softmax function and can be executed in a few steps efficiently.



Fig. 2. Stills from videos compressed at CRF 40.

3.5 Reconstruction Module

We pass the aggregated features F_{agg} to the reconstruction module f_r which produces the residuals that get added to the input image. To match the resolutions, a pixel shuffle [16] is produced at the end of the reconstruction block.

$$RE^{\text{face}} = LQ^{\text{face}} + f_r(F_{agg}) \quad (5)$$

Similar to other papers [1,10] we find producing residuals with the model more effective than directly producing the output. The reconstruction module is composed of 10 shuffle units. Finally, we replace the low-quality face crop with the enhanced face crop RE^{face} to produce the final output RE .

3.6 Enhanced Speed For Subsequent Frames

One integral aspect of the model is the ability to reuse reference features for subsequent frames. Once features have been extracted from the high quality reference images, they can be cached and used throughout the video without having to go through the feature extractor. With these reference features, we can bypass the detector and extraction steps and directly move to the alignment step saving computation over the course of a call.

4 Experiments

4.1 Datasets

Similar to [1] and [10], we use the publicly available DeepFakeDetection dataset [15] to perform our experiments. The dataset is comprised of 363 1080p videos

that were made using 28 paid actors. Out of these videos, we used the videos belonging to the categories "*outside talking still laughing*", "*podium speech happy*" and "*talking against wall*" as these were the ones most similar to a video call setting. The first 22 identities corresponding to 65 videos were used as the training set and the remaining 6 identities corresponding to 17 videos were used as the testing set.

For all videos, we first cropped a 512×512 pixel region around the human subject, extracted 5 frames from the first 12 seconds of the video at a sampling rate of 72 frames (input videos have a frame rate of 24 frames per second) to form our reference images, skipped 3 seconds and then compressed the rest of the video. We chose this policy as it most resembles a real videocall scenario where the high quality frames can be sent early in the stream, with the rest of the stream being compressed. While we extracted 5 reference frames, we only used 3 in our model. Some frames didn't contain the entire face so we extracted 2 more to have more options to choose from.

Our low quality videos were compressed at three different compression rates using two of the most popular video compression standards - H.264 and H.265. We used `ffmpeg` with the `libx264` encoder for H.264 and `libx265` for H.265. The compression rates we chose were a Constant Rate Factor (CRF) of 36, 40 and 44 to mimic a range of low bandwidth settings. For reference, the CRF parameter ranges from 0-51 where 0 is lossless, 18 is considered visually lossless and 51 is the worst quality possible. Generally speaking, a CRF of 17-28 is considered an acceptable range according to the `ffmpeg` video encoding guide. The reason we chose to experiment with H.264 is because it is still a widely adopted codec for videocalls with the greatest support across a range of devices including older Android and iOS phones. H.264 and its variants are also the codecs used by popular videoconferencing applications such as Zoom and Webex. By including H.265 video in our tests, we demonstrate that our model operates reliably irrespective of the codec used.

To speed up training, instead of training on the entire video, we trained on 20 frames from each video sampled at a rate of 10 frames. We also trained our model on a tighter cropped region around the face with a resolution of 256×256 . These were obtained using the aforementioned face detector 3.1.

4.2 Training

We trained our model for 1000 epochs using the Adam optimizer with a learning rate of 10^{-4} and the default $\beta_1 = 0.9$, $\beta_2 = 0.999$. To improve the performance of our model, we performed a number of random augmentations including, random horizontal flip, random vertical flip, and a random affine transformation with a rotation in the range $[-5^\circ, 5^\circ]$, translation along the horizontal and vertical axes in the range $[-2\%, 2\%]$ and scaling in the range of $[0.95, 1.05]$. No shearing was performed. Our model can take any number 1 or more reference images; for our experiments, we use 3 reference images.

We compared our model's performance to the baseline input compressed video. While there are numerous works on face restoration and face enhancement,



Fig. 3. Stills from videos compressed at CRF 44.

to the best of our knowledge, none of these works can run in realtime (at least 24 FPS) on a typical CPU, and thus we didn't evaluate comparisons against them. In this work we are primarily concerned with face enhancement in constrained resource settings.

The recreation loss l_{rec} we optimized for was the Mean Absolute Error which is given by:

$$l_{rec} = \frac{1}{HWC} \|HQ - RE\|_1 \quad (6)$$

where HQ is the high quality ground truth frame and RE is the reconstructed frame. HWC is the height, width and number of channels in HQ and RE . We also performed adversarial training on our model based on the LS-GAN [13] where discriminator loss l_{dis} and generator loss l_{gen} are given by:

$$l_{dis} = \frac{1}{2} \mathbb{E}_{HQ \sim P(HQ)} [(D(HQ) - 1)^2] + \frac{1}{2} \mathbb{E}_{LQ \sim P(LQ)} [D(G(LQ))^2] \quad (7)$$

$$l_{gen} = \frac{1}{2} \mathbb{E}_{LQ \sim P(LQ)} [(D(G(LQ)) - 1)^2] \quad (8)$$

The total loss l_{total} for our GAN based model is given by:

$$l_{total} = l_{rec} + 0.01 * l_{gen} \quad (9)$$

4.3 Quantitative Results

To evaluate our models we considered both distortion and perceptual metrics. For distortion, we measured Peak Signal to Noise Ratio (PSNR), Structural

Similarity (SSIM) and for a perceptual metric, we measured Learned Perceptual Image Patch Similarity (LPIPS) [22]. We used the version of LPIPS that uses the AlexNet [11] backbone. The scores are reported in table 2. All scores reported on a 256×256 face region. Our RTFVE model, when trained solely with reconstruction loss, achieves PSNR improvements of approximately 0.5, 0.8, and 0.76 dB for CRF values 36, 40, and 44 on the H.264 datasets respectively. Similarly, for videos compressed with the H.265 standard, the model yields PSNR gains of about 0.66, 0.73, and 0.72 dB for CRF 36, 40, and 44 respectively. These results confirm that our model delivers consistent performance improvements regardless of the codec used. Similarly, there is also an increase in SSIM. This is impressive given the severity of degradation found in the CRF 40 and 44 videos. Our GAN model, named RTFVE-gan, is not only able to increase PSNR and SSIM but is also able to reduce LPIPS scores which is considered better for this metric. We found that when the model has fewer parameters to train, it becomes harder to lower LPIPS scores so given the low number of parameters in our model, lowering LPIPS is also impressive.

Table 1 shows the speed of our model on CPU compared to other state-of-the-art face restoration models. Our frame rates include the time taken by the face detector model to find the face. With minimal inference optimizations and no quantization our model runs at a median of 24.85 frames per second on a circa 2018 laptop with an Intel® Core™ i7-8550U CPU @ 1.80GHz $\times$ 8. It is reasonable to assume that a typical video call user may have this level of resource or better, and thus in practice our model runs in realtime and is suited for most everyday settings where a GPU or NPU is unavailable. On the other hand, typical face restoration models are not designed for low latency and low compute environments and hence fall well short of the 24 frames per second required for realtime applications. Our model also consumes low memory and is only 190K parameters - less than a megabyte of disk space. When reference features are recalculated for each frame, speed falls to 19.04 fps, which shows how our caching design improves inference speed.

Since our model, to the best of our knowledge, is the only model capable of running in realtime on CPU we couldn't compare our model with other face restoration models. Even classic, handcrafted image processing algorithms such as Non-Local Means Denoising [3], Bilateral Filtering [17] and total variation denoising methods such as Split Bregman [6] and Chambolle [4] do not run in realtime 1. We did however compare against low-pass filters that have lower complexity and can be run in realtime. For this experiment we considered H.264 videos compressed at a CRF of 40. As we can see in table 3, many of these filters produce worse results compared to the given compressed video (baseline). Only the Gaussian Filter was able to marginally improve performance and is well short of our model's output.

4.4 Qualitative Results

Figures 2 and 3 compare quality between the compressed and enhanced frames. These stills were taken from videos compressed using the H.264 standard. The

Table 3. Various realtime methods on H.264 CRF 40 data.

Model	PSNR ($\uparrow$)	SSIM ($\uparrow$)	LPIPS ($\downarrow$)
H.264 (baseline)	30.833	0.8752	0.1273
Mean Filter	28.2071	0.8787	0.2259
Median Filter	30.5624	0.8766	0.1519
Wavelet Denoising	30.848	0.8762	0.1288
Gaussian Filter	30.964	0.8821	0.1623
RTFVE (Ours)	31.6259	0.8961	0.1365

reader is encouraged to view these figures realistically enlarged on a screen. Figure 2 shows the output stills of our model for a video compressed at CRF 40. From the first row, we can see that the input low quality frame has many artifacts including unnatural skin texture and blocking artifacts around the mouth. In the second row we can see that the model was able to alleviate these issues by producing smoother looking skin and has considerably reduced the blocking around the mouth, nose and chin. The output produced is also considerably sharper with the individual parts of the face being more well defined. In figure 3 we can see the model’s performance on a CRF 44 video. Here the input is heavily deformed with large blocking artifacts and speckly skin texture. The model again does a good job smoothing the skin and improving the definition of the face under these difficult conditions. The improved quality is arguably more significant than the improvement in PSNR suggests. A video demo is available at <https://sigport.org/documents/rtfve>.

5 Conclusion

This paper introduces a novel lightweight model for videocall face enhancement. Unlike previous models for face enhancement, our model can run in realtime on typical user devices (e.g. laptop CPU) making it widely applicable. Our model is able to objectively and subjectively improve the video call experience under constrained circumstances where compression rates are high and visual quality is low. It achieves this by leveraging the information contained in high quality reference frames which are very similar to the low quality frames in the input stream. Our model can be seamlessly integrated with current compression standards where the model would be taking as input the output of the decoder.

For future work, we would like to find ways of further improving quality under constrained circumstances. Obtaining better perceptual quality with small models is challenging due to a lack of modeling ability but we seek to find ways to better exploit high quality reference images. We are also interested in building realtime models for larger resolutions.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Agnolucci, L., Galteri, L., Bertini, M., Bimbo, A.D.: Perceptual quality improvement in videoconferencing using keyframes-based gan. *IEEE Transactions on Multimedia* **26**, 339–352 (2024). <https://doi.org/10.1109/TMM.2023.3264882>
2. Bazarevsky, V., Kartynnik, Y., Vakunov, A., Raveendran, K., Grundmann, M.: Blazeface: Sub-millisecond neural face detection on mobile gpus. In: *arxiv.org* (2019), <https://arxiv.org/abs/1907.05047>
3. Buades, A., Coll, B., Morel, J.M.: A non-local algorithm for image denoising. In: 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’05). vol. 2, pp. 60–65 vol. 2 (2005). <https://doi.org/10.1109/CVPR.2005.38>
4. Chambolle, A.: An algorithm for total variation minimization and applications. *Journal of Mathematical Imaging and Vision* **20**, 89–97 (2004). <https://doi.org/10.1023/B:0000011325.36760.1e>
5. Chan, K.C., Wang, X., Yu, K., Dong, C., Loy, C.C.: Basicvsr: The search for essential components in video super-resolution and beyond. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 4947–4956 (June 2021)
6. Getreuer, P.: Rudin-Osher-Fatemi Total Variation Denoising using Split Bregman. *Image Processing On Line* **2**, 74–95 (2012), <https://doi.org/10.5201/ipol.2012.g-tvd>
7. Guan, Z., Xing, Q., Xu, M., Yang, R., Liu, T., Wang, Z.: Mfqc 2.0: A new approach for multi-frame quality enhancement on compressed video. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **43**(3), 949–963 (2021). <https://doi.org/10.1109/TPAMI.2019.2944806>
8. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 770–778 (2016). <https://doi.org/10.1109/CVPR.2016.90>
9. Jiang, Y., Chan, K.C., Wang, X., Loy, C.C., Liu, Z.: Robust reference-based super-resolution via c2-matching. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 2103–2112 (2021). <https://doi.org/10.1109/CVPR46437.2021.00214>
10. Jois, V.R., DiLillo, A., Storer, J.: Reference-based face super-resolution using the spatial transformer. In: *Computer Vision – ACCV 2024: 17th Asian Conference on Computer Vision, Hanoi, Vietnam, December 8–12, 2024, Proceedings, Part IV*. p. 409–425. Springer-Verlag, Berlin, Heidelberg (2024). https://doi.org/10.1007/978-981-96-0911-6_24, https://doi.org/10.1007/978-981-96-0911-6_24
11. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. *Commun. ACM* **60**(6), 84–90 (May 2017). <https://doi.org/10.1145/3065386>, <https://doi.org/10.1145/3065386>
12. Ma, N., Zhang, X., Zheng, H.T., Sun, J.: Shufflenet v2: Practical guidelines for efficient cnn architecture design. In: Ferrari, V., Hebert, M., Sminchisescu, C., Weiss, Y. (eds.) *Computer Vision – ECCV 2018*. pp. 122–138. Springer International Publishing, Cham (2018)
13. Mao, X., Li, Q., Xie, H., Lau, R.Y., Wang, Z., Smolley, S.P.: Least squares generative adversarial networks. In: 2017 IEEE International Conference on Computer Vision (ICCV). pp. 2813–2821 (2017). <https://doi.org/10.1109/ICCV.2017.304>
14. Pouyanfar, S., Sengupta, S., Mohammadi, M., Abraham, E., Bloomquist, B., Dauterman, L., Parikh, A., Lim, S., Sommerlade, E.: Frr-net: A real-time blind

- face restoration and relighting network. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops. pp. 1240–1250 (June 2023)
15. Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., Niessner, M.: Faceforensics++: Learning to detect manipulated facial images. In: 2019 IEEE/CVF International Conference on Computer Vision (ICCV). pp. 1–11 (2019). <https://doi.org/10.1109/ICCV.2019.00009>
 16. Shi, W., Caballero, J., Huszár, F., Totz, J., Aitken, A.P., Bishop, R., Rueckert, D., Wang, Z.: Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 1874–1883 (2016). <https://doi.org/10.1109/CVPR.2016.207>
 17. Tomasi, C., Manduchi, R.: Bilateral filtering for gray and color images. In: Sixth International Conference on Computer Vision (IEEE Cat. No.98CH36271). pp. 839–846 (1998). <https://doi.org/10.1109/ICCV.1998.710815>
 18. Wang, X., Chan, K.C., Yu, K., Dong, C., Change Loy, C.: Edvr: Video restoration with enhanced deformable convolutional networks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops (June 2019)
 19. Wang, X., Li, Y., Zhang, H., Shan, Y.: Towards real-world blind face restoration with generative facial prior. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 9164–9174 (2021). <https://doi.org/10.1109/CVPR46437.2021.00905>
 20. Yang, P., Zhou, S., Tao, Q., Loy, C.C.: Pgdif: Guiding diffusion models for versatile face restoration via partial guidance. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) *Advances in Neural Information Processing Systems*. vol. 36, pp. 32194–32214. Curran Associates, Inc. (2023), https://proceedings.neurips.cc/paper_files/paper/2023/file/661c37f3b098bdee53fd7d9c4ef6964a-Paper-Conference.pdf
 21. Yang, T., Ren, P., Xie, X., Zhang, L.: Gan prior embedded network for blind face restoration in the wild. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 672–681 (June 2021)
 22. Zhang, R., Isola, P., Efros, A.A., Shechtman, E., Wang, O.: The unreasonable effectiveness of deep features as a perceptual metric. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (June 2018)
 23. Zhou, S., Chan, K., Li, C., Loy, C.C.: Towards robust blind face restoration with codebook lookup transformer. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems*. vol. 35, pp. 30599–30611. Curran Associates, Inc. (2022), https://proceedings.neurips.cc/paper_files/paper/2022/file/c573258c38d0a3919d8c1364053c45df-Paper-Conference.pdf